

Министерство науки и высшего образования Российской Федерации

Пензенский государственный университет

Кафедра «Вычислительная техника»

ОТЧЁТ

по лабораторной работе №8

по курсу «Логика и основы алгоритмизации в инженерных задачах»

на тему «Обход графа в ширину»

Выполнили:

студенты группы 24ВВВЗ

Задоркин Тимофей

Комиссаров Андрей

Приняли:

Юрова О.В.

Деев М.В.

Пенза 2025

Цель работы – ознакомиться с алгоритмом обхода графа в ширину и освоить его практическую реализацию.

Общие сведения.

Обход графа в ширину – еще один распространенный способ обхода графов.

Основная идея такого обхода состоит в том, чтобы посещать вершины по уровням удаленности от исходной вершины. Удалённость в данном случае понимается как количество ребер, по которым необходимо прейти до достижения вершины. Например, если для графа на рисунке начать обход из первой вершины, то вершины 3, 6 и 2 будут находиться на уровне удаленности в 1 ребро, а вершины 5 и 4 на уровне удаленности в 2 ребра.

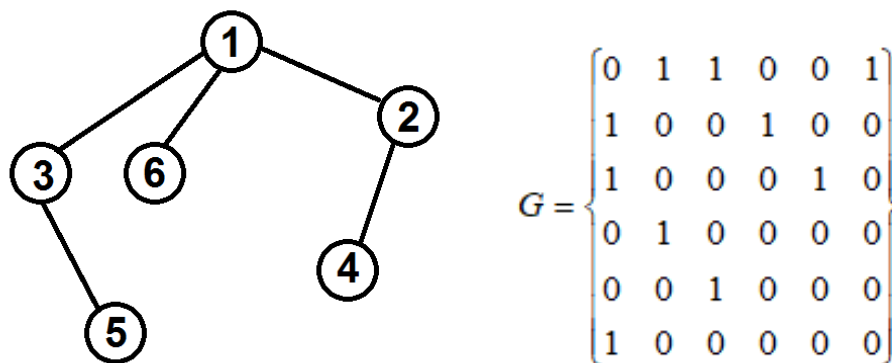


Рисунок – Граф

Тогда при обходе этого графа в ширину, мы сначала посетим вершины первого уровня удаленности (с номерами 2, 3 и 6), и только после того, как закончатся не посещенные вершины на этом уровне, мы перейдем к следующему. На втором уровне мы посетим все вершины, которые удалены от исходной на 2 ребра (вершины 4 и 5).

Так, алгоритм обхода в ширину продолжает осматривать уровень за уровнем, пока не пройдет все доступные вершины.

Чтобы не заходить повторно в уже пройденные вершины, они помечаются, как и в алгоритме обхода в глубину.

Для того, чтобы проход осуществлялся по уровням необходимо хранить информацию о требуемом порядке посещения вершин. Вершины, которые являются ближайшими соседями исходной вершины (из которой начат обход) должны быть посещены раньше, чем соседи соседей и т.д. Такой порядок позволяет задать структура данных «очередь». Просматривая строку матрицы смежности (или список смежности) для текущей вершины мы помещаем всех её ещё не посещенных соседей в очередь. На следующей итерации текущей вершиной становится та, которая стоит в очереди первой и уже её не посещенные соседи будут помещены в очередь. Но место в очереди они займут после тех вершин, которые были помещены туда на предыдущих итерациях.

Таким образом, можно предложить следующую реализацию алгоритма обхода в ширину.

Вход: G – матрица смежности графа.

Выход: номера вершин в порядке их прохождения на экране.

Алгоритм ПОШ

1.1. для всех i положим $NUM[i] = \text{False}$ пометим как "не посещенную";

1.2. **ПОКА** существует "новая" вершина v

1.3. **ВЫПОЛНЯТЬ** BFS (v).

Алгоритм BFS(v):

2.1. Создать пустую очередь $Q = \{\}$;

2.2. Поместить v в очередь $Q.push(v)$;

- 2.3. пометить v как "посещенную" $NUM[v] = \text{True}$;
- 2.4. **ПОКА** $Q \neq \emptyset$ очередь не пуста **ВЫПОЛНЯТЬ**
- 2.5. $v = Q.\text{front}()$ установить текущую вершину;
- 2.6. Удалить первый элемент из очереди $Q.\text{pop}()$;
- 2.7. вывести на экран v ;
- 2.8. **ДЛЯ** $i = 1$ **ДО** size_G **ВЫПОЛНЯТЬ**
- 2.9. **ЕСЛИ** $G(v,i) = 1$ **И** $NUM[i] = \text{False}$
- 2.10. **ТО**
- 2.11. Поместить i в очередь $Q.\text{push}(i)$;
- 2.12. пометить v как "посещенную" $NUM[v] = \text{True}$;

Задание 1

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа G . Выведите матрицу на экран.
2. Для сгенерированного графа осуществите процедуру обхода в ширину, реализованную в соответствии с приведенным выше описанием. При реализации алгоритма в качестве очереди используйте класс **queue** из стандартной библиотеки C++.
- 3.* Реализуйте процедуру обхода в ширину для графа, представленного списками смежности.

Задание 2*

1. Для матричной формы представления графов реализуйте алгоритм обхода в ширину с использованием очереди, построенной на основе структуры данных «список», самостоятельно созданной в лабораторной работе № 3.
2. Оцените время работы двух реализаций алгоритмов обхода в ширину (использующего стандартный класс **queue** и использующего очередь, реализованную самостоятельно) для графов разных порядков.

Результат работы программы

```
Input number of vertices: 6
Adjacency Matrix:
  0  1  1  1  0  0
  1  0  1  0  1  0
  1  1  0  1  0  1
  1  0  1  0  0  1
  0  1  0  0  0  0
  0  0  1  1  0  0

Adjacency Lists:
Vertex 0: 3 -> 2 -> 1 -> NULL
Vertex 1: 4 -> 2 -> 0 -> NULL
Vertex 2: 5 -> 3 -> 1 -> 0 -> NULL
Vertex 3: 5 -> 2 -> 0 -> NULL
Vertex 4: 1 -> NULL
Vertex 5: 3 -> 2 -> NULL

Input the start vertex: 0

=== BFS with Matrix (queue) ===
Path:  0  1  2  3  4  5

=== BFS with Matrix (Custom ) ===
Path:  0  1  2  3  4  5

=== BFS with Lists (queue) ===
Path:  0  3  2  1  5  4

=== BFS with Lists (Custom ) ===
Path:  0  3  2  1  5  4

+-----+-----+
| Algorithm          | Time (seconds) |
+-----+-----+
| Matrix (queue)     | 0.000754       |
| Matrix (Custom)    | 0.000268       |
| AdjList (queue)    | 0.000433       |
| AdjList (Custom)   | 0.000335       |
+-----+-----+
```

Рисунок 1 – 6 вершин

```

Input number of vertices: 10
Adjacency Matrix:
  0  1  1  0  0  1  1  1  0  1
  1  0  1  1  1  0  0  0  0  0
  1  1  0  0  1  1  1  1  1  0
  0  1  0  0  1  0  0  1  1  1
  0  1  1  1  0  0  0  0  1  0
  1  0  1  0  0  0  0  0  1  0
  1  0  1  0  0  0  0  0  1  1
  1  0  1  1  0  0  0  0  1  1
  0  0  1  1  1  1  1  1  0  1
  1  0  0  1  0  0  1  1  1  0

Adjacency Lists:
Vertex 0: 9 -> 7 -> 6 -> 5 -> 2 -> 1 -> NULL
Vertex 1: 4 -> 3 -> 2 -> 0 -> NULL
Vertex 2: 8 -> 7 -> 6 -> 5 -> 4 -> 1 -> 0 -> NULL
Vertex 3: 9 -> 8 -> 7 -> 4 -> 1 -> NULL
Vertex 4: 8 -> 3 -> 2 -> 1 -> NULL
Vertex 5: 8 -> 2 -> 0 -> NULL
Vertex 6: 9 -> 8 -> 2 -> 0 -> NULL
Vertex 7: 9 -> 8 -> 3 -> 2 -> 0 -> NULL
Vertex 8: 9 -> 7 -> 6 -> 5 -> 4 -> 3 -> 2 -> NULL
Vertex 9: 8 -> 7 -> 6 -> 3 -> 0 -> NULL

Input the start vertex: 0

=== BFS with Matrix (queue) ===
Path:  0  1  2  5  6  7  9  3  4  8

=== BFS with Matrix (Custom ) ===
Path:  0  1  2  5  6  7  9  3  4  8

=== BFS with Lists (queue) ===
Path:  0  9  7  6  5  2  1  8  3  4

=== BFS with Lists (Custom ) ===
Path:  0  9  7  6  5  2  1  8  3  4

+-----+-----+
| Algorithm | Time (seconds) |
+-----+-----+
| Matrix (queue) | 0.000578 |
| Matrix (Custom) | 0.000507 |
| AdjList (queue) | 0.000445 |
| AdjList (Custom) | 0.000430 |
+-----+-----+

```

Рисунок 2 – 10 вершин

```

Input number of vertices: 50
=== BFS with Matrix (queue) ===
Path:  0  4  6 12 13 14 15 16 17 19 21 23 24 25 28 31 33 34 36 37 3
8 40 41 47 48 49  1  3  5  8 22 26 29 32 39 46  2  7 11 18 20 35 42
44  9 10 27 43 45 30

=== BFS with Matrix (Custom ) ===
Path:  0  4  6 12 13 14 15 16 17 19 21 23 24 25 28 31 33 34 36 37 3
8 40 41 47 48 49  1  3  5  8 22 26 29 32 39 46  2  7 11 18 20 35 42
44  9 10 27 43 45 30

=== BFS with Lists (queue) ===
Path:  0 49 48 47 41 40 38 37 36 34 33 31 28 25 24 23 21 19 17 16 1
5 14 13 12  6  4 46 45 44 43 39 32 29 27 22 11 10  5  2 42 35 30 26
18  9  8  7  3  1 20

=== BFS with Lists (Custom ) ===
Path:  0 49 48 47 41 40 38 37 36 34 33 31 28 25 24 23 21 19 17 16 1
5 14 13 12  6  4 46 45 44 43 39 32 29 27 22 11 10  5  2 42 35 30 26
18  9  8  7  3  1 20

+-----+-----+
| Algorithm | Time (seconds) |
+-----+-----+
| Matrix (queue) | 0.002645 |
| Matrix (Custom) | 0.002292 |
| AdjList (queue) | 0.003042 |
| AdjList (Custom) | 0.002174 |
+-----+-----+

```

Рисунок 3 – 50 вершин

```

Input number of vertices: 100
=== BFS with Matrix (queue) ===
Path:  0  1  2  3  4  5  6  8 10 13 14 19 20 21 22 23 24 28 29 30 31 34 35 36 38 39 40 43 47 48 49 54 55 56 58 59 60 61
65 66 67 69 70 73 76 77 80 86 87 88 89 90 91 92 94 96 99  7  9 11 15 17 18 25 27 32 33 50 57 68 71 75 78 79 83 85 93 95
16 26 37 41 44 45 53 72 81 84 97 98 46 52 62 63 64 12 42 74 82 51

=== BFS with Matrix (Custom ) ===
Path:  0  1  2  3  4  5  6  8 10 13 14 19 20 21 22 23 24 28 29 30 31 34 35 36 38 39 40 43 47 48 49 54 55 56 58 59 60 61
65 66 67 69 70 73 76 77 80 86 87 88 89 90 91 92 94 96 99  7  9 11 15 17 18 25 27 32 33 50 57 68 71 75 78 79 83 85 93 95
16 26 37 41 44 45 53 72 81 84 97 98 46 52 62 63 64 12 42 74 82 51

=== BFS with Lists (queue) ===
Path:  0 99 96 94 92 91 90 89 88 87 86 80 77 76 73 70 69 67 66 65 61 60 59 58 56 55 54 49 48 47 43 40 39 38 36 35 34 31
30 29 28 24 23 22 21 20 19 14 13 10  8  6  5  4  3  2  1 98 97 95 84 81 79 68 64 63 51 42 41 26 25 17 16  9 93 85 83 82
78 71 62 45 44 33 15 11  7 75 72 46 37 32 74 53 52 12 57 50 27 18

=== BFS with Lists (Custom ) ===
Path:  0 99 96 94 92 91 90 89 88 87 86 80 77 76 73 70 69 67 66 65 61 60 59 58 56 55 54 49 48 47 43 40 39 38 36 35 34 31
30 29 28 24 23 22 21 20 19 14 13 10  8  6  5  4  3  2  1 98 97 95 84 81 79 68 64 63 51 42 41 26 25 17 16  9 93 85 83 82
78 71 62 45 44 33 15 11  7 75 72 46 37 32 74 53 52 12 57 50 27 18

+-----+-----+
| Algorithm | Time (seconds) |
+-----+-----+
| Matrix (queue) | 0.005402 |
| Matrix (Custom) | 0.004386 |
| AdjList (queue) | 0.004480 |
| AdjList (Custom) | 0.005108 |
+-----+-----+

```

Рисунок 4 – 100 вершин

```

Graph type (0 - undirected, 1 - directed): 1
Input number of vertices: 7
Adjacency Matrix:
  0  1  0  0  1  0  0
  1  0  0  0  1  1  0
  0  0  0  0  1  0  0
  1  1  0  0  1  1  1
  1  0  1  1  0  0  1
  0  1  0  0  0  0  1
  0  1  0  1  1  1  0

```

```

Adjacency Lists:
Vertex 0: 4 -> 1 -> NULL
Vertex 1: 5 -> 4 -> 0 -> NULL
Vertex 2: 4 -> NULL
Vertex 3: 6 -> 5 -> 4 -> 1 -> 0 -> NULL
Vertex 4: 6 -> 3 -> 2 -> 0 -> NULL
Vertex 5: 6 -> 1 -> NULL
Vertex 6: 5 -> 4 -> 3 -> 1 -> NULL

```

```

Input the start vertex: 0

```

```

=== BFS with Matrix (STL queue) ===
Path:  0  1  4  5  2  3  6

```

```

=== BFS with Matrix (Custom Queue) ===
Path:  0  1  4  5  2  3  6

```

```

=== BFS with Lists (STL queue) ===
Path:  0  4  1  6  3  2  5

```

```

=== BFS with Lists (Custom Queue) ===
Path:  0  4  1  6  3  2  5

```

Algorithm	Time (seconds)
Matrix (queue)	0.000629
Matrix (Custom)	0.000349
AdjList (queue)	0.000503
AdjList (Custom)	0.000497

Рисунок 5 – ориентированный граф

Вывд: в ходе выполнения лабораторной работы ознакомились с алгоритмом обхода графа в ширину и освоили его практическую реализацию.

Приложение – Листинг программы

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
#include <time.h>
#include <queue>
#include <windows.h>

using namespace std;

struct Node {
    int vertex;
    Node* next;
};

struct QueueNode {
    int data;
    QueueNode* next;
};

struct Queue {
    QueueNode* front;
    QueueNode* rear;
};

Queue* createQueue() {
    Queue* q = (Queue*)malloc(sizeof(Queue));
    q->front = q->rear = NULL;
    return q;
}

int isEmpty(Queue* q) {
    return q->front == NULL;
}

void enqueue(Queue* q, int data) {
    QueueNode* newNode = (QueueNode*)malloc(sizeof(QueueNode));
    newNode->data = data;
    newNode->next = NULL;

    if (q->rear == NULL) {
        q->front = q->rear = newNode;
        return;
    }

    q->rear->next = newNode;
    q->rear = newNode;
}

int dequeue(Queue* q) {
    if (isEmpty(q)) {
```

```

        return -1;
    }

    QueueNode* temp = q->front;
    int data = temp->data;

    q->front = q->front->next;

    if (q->front == NULL) {
        q->rear = NULL;
    }

    free(temp);
    return data;
}

void freeQueue(Queue* q) {
    while (!isEmpty(q)) {
        dequeue(q);
    }
    free(q);
}

void BFS_Matrix_STL(int** G, int numG, int* visited, int s) {
    queue<int> q;
    int v;

    visited[s] = 1;
    q.push(s);
    while (!q.empty())
    {
        v = q.front();
        q.pop();

        printf("%3d", v);

        for (int i = 0; i < numG; i++) {
            if (G[v][i] == 1 && visited[i] == 0) {
                q.push(i);
                visited[i] = 1;
            }
        }
    }
}

void BFS_Matrix_CustomQueue(int** G, int numG, int* visited, int
s) {
    Queue* q = createQueue();
    int v;

    visited[s] = 1;
    enqueue(q, s);

```

```

        while (!isQueueEmpty(q)) {
            v = dequeue(q);

            printf("%3d", v);

            for (int i = 0; i < numG; i++) {
                if (G[v][i] == 1 && visited[i] == 0) {
                    enqueue(q, i);
                    visited[i] = 1;
                }
            }

            freeQueue(q);
        }
    }

void BFS_AdjList_STL(Node** adjList, int numG, int* visited, int
s) {
    queue<int> q;
    int v;

    visited[s] = 1;
    q.push(s);

    while (!q.empty()) {
        v = q.front();
        q.pop();

        printf("%3d", v);

        Node* current = adjList[v];
        while (current != NULL) {
            int neighbor = current->vertex;
            if (visited[neighbor] == 0) {
                q.push(neighbor);
                visited[neighbor] = 1;
            }
            current = current->next;
        }
    }
}

void BFS_AdjList_CustomQueue(Node** adjList, int numG, int*
visited, int s) {
    Queue* q = createQueue();
    int v;

    visited[s] = 1;
    enqueue(q, s);

    while (!isQueueEmpty(q)) {
        v = dequeue(q);
    }
}

```

```

        printf("%3d", v);

        Node* current = adjList[v];
        while (current != NULL) {
            int neighbor = current->vertex;
            if (visited[neighbor] == 0) {
                enqueue(q, neighbor);
                visited[neighbor] = 1;
            }
            current = current->next;
        }
    }

    freeQueue(q);
}

Node** createAdjListFromMatrix(int** matrix, int numG) {
    Node** adjList = (Node**)malloc(numG * sizeof(Node*));

    for (int i = 0; i < numG; i++) {
        adjList[i] = NULL;
        for (int j = 0; j < numG; j++) {
            if (matrix[i][j] == 1 && i != j) {
                Node* newNode = (Node*)malloc(sizeof(Node));
                newNode->vertex = j;
                newNode->next = adjList[i];
                adjList[i] = newNode;
            }
        }
    }

    return adjList;
}

void printMatrix(int** G, int numG) {
    printf("Adjacency Matrix:\n");
    for (int i = 0; i < numG; i++) {
        for (int j = 0; j < numG; j++) {
            printf("%3d", G[i][j]);
        }
        printf("\n");
    }
}

void printAdjList(Node** adjList, int numG) {
    printf("\nAdjacency Lists:\n");
    for (int i = 0; i < numG; i++) {
        printf("Vertex %d: ", i);
        Node* current = adjList[i];
        while (current != NULL) {
            printf("%d -> ", current->vertex);
            current = current->next;
        }
    }
}

```

```

        printf("NULL\n");
    }
}

void initVisited(int* visited, int numG) {
    for (int i = 0; i < numG; i++) {
        visited[i] = 0;
    }
}

void freeAdjList(Node** adjList, int numG) {
    for (int i = 0; i < numG; i++) {
        Node* current = adjList[i];
        while (current != NULL) {
            Node* temp = current;
            current = current->next;
            free(temp);
        }
    }
    free(adjList);
}

double measureTime(void (*bfs_func)(int**, int, int*, int),
int** graph, int numG, int* visited, int start) {
    LARGE_INTEGER frequency, start_time, end_time;
    QueryPerformanceFrequency(&frequency);
    QueryPerformanceCounter(&start_time);

    bfs_func(graph, numG, visited, start);

    QueryPerformanceCounter(&end_time);
    return (double)(end_time.QuadPart - start_time.QuadPart) /
frequency.QuadPart;
}

double measureTimeAdjList(void (*bfs_func)(Node**, int, int*,
int), Node** adjList, int numG, int* visited, int start) {
    LARGE_INTEGER frequency, start_time, end_time;
    QueryPerformanceFrequency(&frequency);
    QueryPerformanceCounter(&start_time);

    bfs_func(adjList, numG, visited, start);

    QueryPerformanceCounter(&end_time);
    return (double)(end_time.QuadPart - start_time.QuadPart) /
frequency.QuadPart;
}

int main() {
    int** G;
    Node** adjList;
    int numG, current;
    int* visited;

```

```

printf("Input number of vertices: ");
scanf("%d", &numG);

visited = (int*)malloc(numG * sizeof(int));
G = (int**)malloc(numG * sizeof(int*));
for (int i = 0; i < numG; i++)
    G[i] = (int*)malloc(numG * sizeof(int));

srand(time(NULL));

for (int i = 0; i < numG; i++) {
    visited[i] = 0;
    for (int j = 0; j < numG; j++) {
        if (i == j) {
            G[i][j] = 0;
        }
        else {
            G[i][j] = rand() % 2;
            if (i > j) {
                G[i][j] = G[j][i];
            }
        }
    }
}

printMatrix(G, numG);

adjList = createAdjListFromMatrix(G, numG);
printAdjList(adjList, numG);

printf("\nInput the start vertex: ");
scanf("%d", &current);

double time_matrix_queue, time_matrix_custom,
time_list_queue, time_list_custom;

printf("\n=== BFS with Matrix (queue) ===\n");
printf("Path: ");
initVisited(visited, numG);
time_matrix_queue = measureTime(BFS_Matrix_STL, G, numG,
visited, current);
printf("\n");

printf("\n=== BFS with Matrix (Custom ) ===\n");
printf("Path: ");
initVisited(visited, numG);
time_matrix_custom = measureTime(BFS_Matrix_CustomQueue, G,
numG, visited, current);
printf("\n");

printf("\n=== BFS with Lists (queue) ===\n");
printf("Path: ");

```

```

    initVisited(visited, numG);
    time_list_queue = measureTimeAdjList(BFS_AdjList_STL,
adjList, numG, visited, current);
    printf("\n");

    printf("\n=== BFS with Lists (Custom ) ===\n");
    printf("Path: ");
    initVisited(visited, numG);
    time_list_custom =
measureTimeAdjList(BFS_AdjList_CustomQueue, adjList, numG,
visited, current);
    printf("\n");

    printf("\n+-----+-----+\n");
    printf("| Algorithm                | Time (seconds)   |\n");
    printf("+-----+-----+\n");
    printf("| Matrix (queue)            | %14.6f   |\n",
time_matrix_queue);
    printf("| Matrix (Custom)          | %14.6f   |\n",
time_matrix_custom);
    printf("| AdjList (queue)          | %14.6f   |\n",
time_list_queue);
    printf("| AdjList (Custom)         | %14.6f   |\n",
time_list_custom);
    printf("+-----+-----+\n");

    free(visited);
    for (int i = 0; i < numG; i++)
        free(G[i]);
    free(G);
    freeAdjList(adjList, numG);

    printf("\nPress any key to exit...");
    _getch();
    return 0;
}

```